Sprint 05 - Drupal Development Guide

Stage PFE – Entreprise **VOID**

Stagiaire : Hibat Allah Rguiti

9 mars 2026

Chapitre 1

Day 1 : Module Development Basics

1.1. Pourquoi créer un module personnalisé ?

Un module personnalisé permet de :

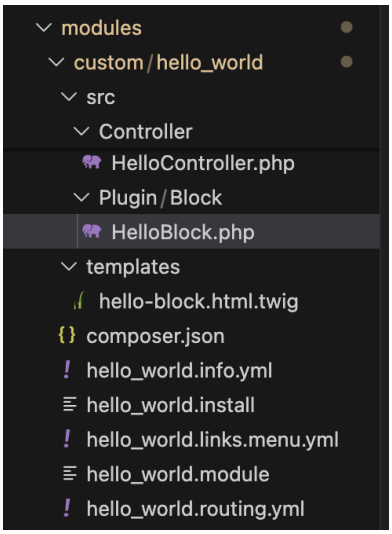
— Modifier un comportement existant sans toucher au code original.

— Intercepter des événements Drupal pour exécuter des actions spécifiques.

— Ajouter de nouvelles fonctionnalités (routes, contrôleurs, services).

1.2. Structure d'un module personnalisé

Un module personnalisé Drupal respecte généralement l'organisation suivante :



1.2.1. Fichiers de configuration du module

Le module Drupal est configuré via plusieurs fichiers YAML :

— `.info.yml` : configure le module auprès de Drupal avec son nom, sa description, la version minimale du core (`core_version_requirement`) et ses dépendances éventuelles.

— **Fichiers dans** `config/install/` : définissent la configuration par défaut. Par exemple, `hello_world.settings.yml` peut contenir :

```
hello:
  name: 'Hank Williams'
```

Ces valeurs sont chargées lors de l'installation du module et peuvent être utilisées dans les blocks ou autres composants.

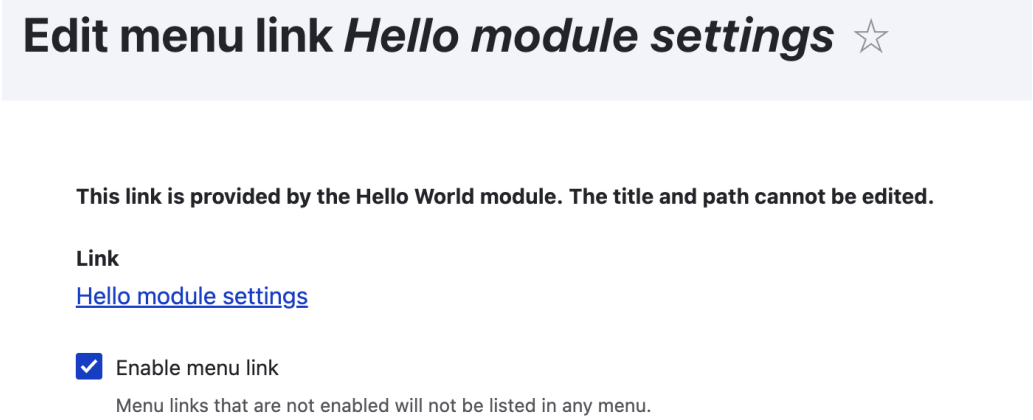
1.3. Menu Links et catégories

1.3.1. Création d'un lien de menu via YAML

Exemple d'un lien de menu module-managed :

```
Drupal Links Menu - Drupal configuration for menu links (drupal-links-menu.json)
hello_world.admin:
  title: 'Hello module settings'
  description: 'example of how to make an admin settings page link'
  route_name: hello_world.content
  parent: system.admin_config_development
  weight: 100
```

1.3.2. Message "cannot be edited"



Les liens créés via YAML sont considérés comme **module-managed** :

- Ils ne peuvent pas être modifiés via l'interface Drupal.
- Le message affiché lors d'une tentative de modification :
  "This link is provided by the XXX module. The title and path cannot be edited."

1.3.3. Hook `hello_world_install()`

Pour créer un lien de menu via code PHP lors de l'installation du module :

```php
hello_world.install U ×
web > modules > custom > hello_world > hello_world.install
<?php

use Drupal\menu_link_content\Entity\MenuLinkContent;

/**
 * Implémentation de hook_install().
 */
function hello_world_install()
{
  $menu_link = MenuLinkContent::create([
    'title' => 'Hello World Page',
    'link' => ['uri' => 'internal:/hello'],
    'menu_name' => 'admin',
    'weight' => 0,
  ]);
  $menu_link->save();
}
```

1.3.4. Création d'une catégorie pour le menu

Il est possible de créer une catégorie pour regrouper les liens d'administration :

```yaml
system.admin_config_helloworld:
  path: '/admin/config/helloworld'
  defaults:
    _controller: '\Drupal\hello_world\Controller\HelloController::systemAdminMenuBlockPage
    _title: 'My Module'
  requirements:
    _permission: 'administer site configuration'
```

```yaml
hello_world.admin3:
  title: 'Hello module'
  description: 'My custom category'
  route_name: system.admin_config_helloworld
  parent: system.admin_config
  weight: 100
```

Hello module ☑ Edit

1.4. Création d'un block personnalisé

1.4.1. Fichier `HelloBlock.php`

Le block est défini comme un plugin dans `src/Plugin/Block/` :

```php
HelloBlock.php U ×
web > modules > custom > hello_world > src > Plugin > Block > HelloBlock.php
#[Block(
    id: "hello_block",
    admin_label: new TranslatableMarkup("Hello block"),
    category: new TranslatableMarkup("Hello World")
)]

class HelloBlock extends BlockBase
{

    /**
     * {@inheritdoc}
     */
    public function build()
    {
        return [
            '#theme' => 'hello_block',
            '#custom_data' => ['name' => 'Hiba', 'age' => 23],
            '#custom_string' => 'Hello Block',
        ];
    }
}
```

1.4.2. Template Twig associée

Le fichier `templates/hello-block.html.twig` permet de thématiser le block :

```
hello-block.html.twig U ×
web > modules > custom > hello_world > templates > hello-block.html.twig > ...
1   {# Output string variable. #}
2     <p>{{ custom_string }}</p>
3
4   {# Output values from the array. #}
5     <strong>{{ custom_data.name }}:</strong> {{ custom_data.age }}<br>
6
7   {# Output array in for loop. #}
8   {% for key,value in custom_data %}
9     <strong>{{ key }}:</strong> {{ value }}<br>
10  {% endfor %}
```

— La méthode `build()` retourne un `RenderArray` qui sera transformé en HTML.
— La clé `#theme` relie le block à la template Twig.
— La méthode `defaultConfiguration()` récupère les valeurs par défaut depuis les fichiers YAML.

Hello block

Hello Block

Hiba: 23
name: Hiba
age: 23

1.5. Questions

1.5.1. Uninstall the modules, and install them again, what happens ?

Lorsqu'un module est désinstallé, Drupal supprime ses configurations de la base de données (menus, blocks, routes, paramètres), mais les fichiers restent sur le disque. En réinstallant le module, Drupal lit les fichiers YAML de `config/install/` et restaure les valeurs par défaut, permettant de tester le module ou de rétablir un état propre.

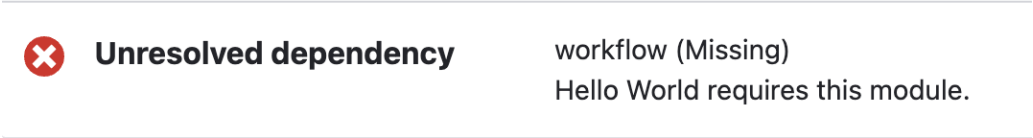
1.5.2. Make change to core_version_requirement to a lower version, what happens ?

Incompatible module

The following module is installed, but it is incompatible with Drupal 11.3.3:
- Hello World

Modifier `core_version_requirement` pour une version plus basse indique que le module peut fonctionner sur un core plus ancien. Cela peut provoquer des erreurs si certaines fonctionnalités du core manquent. Drupal laisse l'installation, mais des warnings peuvent apparaître dans les logs.

1.5.3. Add the module workflows as a dependency to your dependencies, what happens ?



Ajouter `workflows` dans `dependencies:` oblige Drupal à vérifier que ce module est activé avant l'installation. Si le module requis n'est pas actif, l'installation est bloquée et un message s'affiche.

1.5.4. Delete your modules or move them to another directory, what happens ? what message appear in drupal ? what the log says drush ws ?

Si un module custom est supprimé ou déplacé :

- Drupal détecte le module manquant et ne l'affiche plus comme activable.
- Les blocks, menus et routes liés restent en base, mais ne sont plus utilisables.
- `drush ws` affiche des warnings correspondants.

```
112   03/Mar 07:15   config_schema   Warning    No schema for
                                                hello_world.settings. These
                                                errors mean there is
                                                configuration that does not
                                                comply with its schema. This
                                                is not a fatal error, but it
                                                is recommended to fix these
                                                issues. For
```

Chapitre 2

Day 2 : Routes, Controllers, Services & Dependency Injection

2.1. Services

Drupal utilise un système de services et un *container* (hérité de Symfony) pour organiser et fournir toutes les fonctionnalités internes que ton module peut utiliser.

Utilité : utiliser un service te permet de ne pas « réinventer la roue » — Drupal fournit déjà ces objets prêts à l'emploi.

2.1.1. Service Container

Drupal possède un *service container*, une sorte de registre central où tous les services sont enregistrés.

Rôle du container :

— Instancier un service quand il est demandé,
— Gérer les dépendances internes (un service utilisé par un autre),
— Optimiser les performances en instanciant les services une seule fois.

2.1.2. Comment utiliser les services dans un module

Accès via le container

```
\Drupal::service('nom_du_service');
```

Permet d'obtenir une instance d'un service depuis du code procédural ou n'importe où dans ton module.

Injection de dépendances (meilleure pratique)

Dans une classe (par exemple un controller ou un block), on peut demander à Drupal d'injecter automatiquement les services dont tu as besoin via le constructeur. C'est plus propre et facilite le test et la maintenance du code.

2.1.3. Définir son propre service

Si un module a par exemple une logique métier complexe (ex : intégration d'une API externe), on peut :

1. Créer une interface qui décrit ce que fait ton service,
2. Créer une classe qui implémente cette interface,
3. Déclarer ce service dans un fichier `MODULE.services.yml`.

Pourquoi les services sont importants

— Ils séparent la logique métier du code d'intégration avec Drupal,
— Ils rendent ton code plus réutilisable, testable et maintenable,
— Ils permettent au container de gérer les dépendances complexes sans effort.

2.1.4. Exemples de services existants

C'est possible de lister les services via l'API ou avec Drush :

```
drush devel:services
```

```
void@192 first_module % ./vendor/bin/drush devel:services
 ------------------------------------------------------------------------
---
  Id

 ------------------------------------------------------------------------
---
  .service_locator.tRiOYIx

  Drupal\Component\DependencyInjection\ReverseContainer

  Drupal\Core\Access\CheckProviderInterface

  Drupal\Core\Breadcrumb\BreadcrumbPreprocess

  Drupal\Core\Config\ConfigImporterFactory

  Drupal\Core\Config\StorageManagerInterface

  Drupal\Core\Database\EventSubscriber\StatementExecutionSubscriber

  Drupal\Core\Datetime\DatePreprocess
```

2.2. Création de custom service

1. Définir l'interface du service (`ForecastClientInterface`)

```php
interface ForecastClientInterface {

  /**
   * Get the current forecast.
   *
   * @param string $url
   *   URL to use to retrieve forecast data.
   *
   * @return array|null
   *   An array containing the formatted data for the forecast, or null.
   */
  public function getForecastData(string $url): ?array;

}
```

FIGURE 2.1 – Forecast Interface

2. Créer la classe de service (`ForecastClient`)

```php
  public function __construct(ClientInterface $httpClient, LoggerChannelFactoryInterface $logger_factory)
  {
      $this->httpClient = $httpClient;
      $this->logger = $logger_factory->get('anytown');
  }

  /**
   * {@inheritDoc}
   */
  public function getForecastData(string $url): ?array
  {
      try {
          $response = $this->httpClient->request('GET', $url);
          $json = json_decode($response->getBody()->getContents());
      } catch (GuzzleException $e) {
          $this->logger->warning($e->getMessage());
          return NULL;
      }

      $forecast = [];
      foreach ($json->list as $day) {
```

FIGURE 2.2 – Forecast Client Implementation

3. Déclarer le service dans `*.services.yml`

```yaml
services:
  anytown.forecast_client:
    class: Drupal\anytown\ForecastClient
    arguments:
      - '@http_client'
      - '@logger.factory'
  Drupal\anytown\ForecastClientInterface:
    alias: anytown.forecast_client
```

FIGURE 2.3 – Forecast Service

4. Utiliser le service dans un controller via __construct() et create()

```php
public function __construct(ForecastClientInterface $forecast_client)
{
    $this->forecastClient = $forecast_client;
}


public static function create(ContainerInterface $container)
{
    return new self(
        $container->get('anytown.forecast_client')
    );
}


public function build(string $style): array
{
    $style = (in_array($style, ['short', 'extended'])) ? $style : 'short';
    $url = 'https://raw.githubusercontent.com/DrupalizeMe/module-developer-guide-
```

FIGURE 2.4 – Controller utilisant le service

Weather at the market

Check out this weekend's weather forecast and come prepared. The market is mostly outside, and takes place rain or shine.

- Friday will be *light snow* with a high of 43 and a low of 30.
- Saturday will be *scattered clouds* with a high of 47 and a low of 29.
- Sunday will be *scattered clouds* with a high of 47 and a low of 41.

]

2.3. Themeable Output

Le code qui produit du contenu (pages, blocs, etc.) est séparé de la manière dont ce contenu est affiché (HTML/CSS).

```php
$build = [
    'weather_intro' => [
        '#markup' => "<p>Check out this weekend's weather forecast and come prepared. The market is mostly outside, and takes place rain or shine.</p>",
    ],
    'weather_forecast' => $weather_forecast,
    'weather_closures' => [
        '#theme' => 'item_list',
        '#title' => 'Weather related closures',
        '#items' => [
            'Ice rink closed until winter – please stay off while we prepare it.',
            'Parking behind Apple Lane is still closed from all the rain last weekend.',
        ],
    ],
];

return $build;
```

FIGURE 2.5 – Refactor controller using renderable array

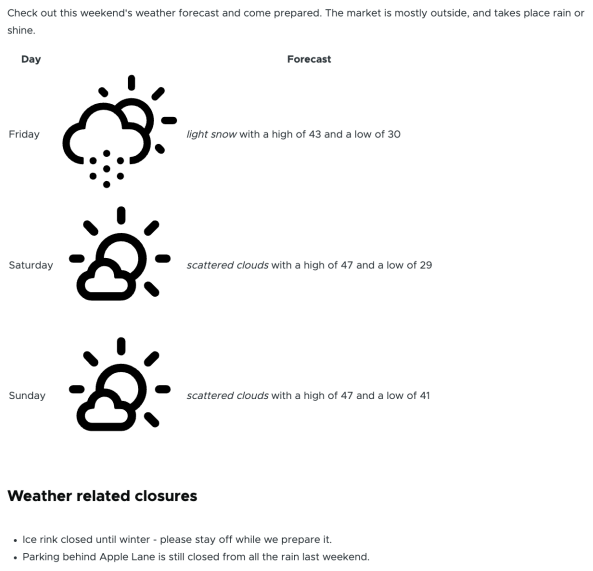


FIGURE 2.6 – Résultat de renderable array

2.4. Add a Template File

Pour afficher du contenu personnalisé dans Drupal, on peut créer un fichier **template Twig**. Le template est associé à un **controller** ou à un hook de thème, permettant de séparer la logique PHP de l'affichage.

```
$build = [
    '#theme' => 'weather_page',
    '#weather_intro' => [
        '#markup' => "<p>Check out this weekend's weather forecast and come prepared. The
    ],
    '#weather_forecast' => $weather_forecast,
    '#short_forecast' => $short_forecast,
    '#weather_closures' => [
        '#theme' => 'item_list',
        '#title' => 'Weather related closures',
        '#items' => [
            'Ice rink closed until winter - please stay off while we prepare it.',
            'Parking behind Apple Lane is still closed from all the rain last weekend.',
        ],
    ],
];
```

FIGURE 2.7 – Controller

```php
class AnytownTheme
{

    /**
     * Implements hook_theme().
     */
    #[Hook('theme')]
    public function theme(): array
    {
        return [
            'weather_page' => [
                'variables' => [
                    'weather_intro' => '',
                    'weather_forecast' => '',
                    'short_forecast' => '',
                    'weather_closures' => '',
                ],
            ],
        ];
    }
}
```

```twig
<div class="weather_page">
  <div class="weather_page--messages">
    {{ weather_intro }}
  </div>
  <div class="weather_page--forecast">
    <h2>Weekend forecast</h2>
    <div class="short">
      {{ short_forecast }}
    </div>
    <div class="long">
      {{ weather_forecast }}
    </div>
  </div>
  {% if weather_closures is not null %}
    <div class="weather_page--closures">
      {{ weather_closures }}
    </div>
  {% endif %}
</div>
```

FIGURE 2.8 – Association entre le hook de thème et le template Twig

2.5. Asset Libraries

Drupal gère les fichiers CSS et JS via les **Asset Libraries**. On définit une bibliothèque dans un fichier YAML et on peut l'attacher aux pages, blocks ou templates.

```yaml
forecast:
  css:
    component:
      css/forecast.css: {}
  js:
    js/forecast.js: {}
  dependencies:
    - core/drupal.once
```

FIGURE 2.9 – Asset Library

2.6. Questions

— **How do I control or sort the menus (weight) ?**

Dans Drupal, les menus sont définis dans les fichiers `*.links.menu.yml`, en associant

chaque lien à une route. L'ordre des liens est contrôlé grâce à la propriété `weight` (plus le poids est bas, plus le lien apparaît en haut).

— **How do I set up child menus ?**
Pour créer des sous-menus, on définit un lien dans le fichier `.links.menu.yml` et on renseigne la propriété `parent` avec l'ID du lien parent. Drupal génère alors automatiquement la structure hiérarchique.

— **How do I retrieve a query string (not to confuse with a parameter) in a Controller ?**
Dans un controller Drupal, les valeurs de la query string (après le ?) peuvent être récupérées via l'objet `$request` : `$request->query->get('parameter_name')`. Cela diffère des paramètres de route.

— **What is Guzzle and Logger ?**
Guzzle et Logger sont des services prédéfinis dans Drupal. Guzzle est un client HTTP pour faire des requêtes vers des APIs externes, tandis que Logger sert à enregistrer des messages dans le système de logs. Les deux sont accessibles via leurs interfaces (`ClientInterface` et `LoggerInterface`) avec des implémentations concrètes dans Drupal.

— **Use Logger to log some messages in your Controller, where do these messages appear ?**
Les messages enregistrés avec Logger n'apparaissent pas dans la console du navigateur. Ils sont sauvegardés dans le système de logs de Drupal (Watchdog) et peuvent être consultés à `/admin/reports/dblog` ou via Drush avec `drush ws`.

— **Where can I find the services defined by Drupal core ?**
Les services du core sont définis dans `core.services.yml`. Les modules custom peuvent également définir leurs propres services dans `*.services.yml`. Drupal charge tous ces fichiers pour construire le container de services.

— **How do you inject other needed services into your service ?**
Pour utiliser d'autres services dans votre service, on peut les injecter via le constructeur (recommandé) ou y accéder directement avec `Drupal::service()`. L'injection par constructeur est préférable car elle rend le code plus testable, découplé et conforme aux bonnes pratiques.

— **How do you return a twig template in a Controller ?**
Dans un controller Drupal, on retourne un template Twig en utilisant un render array. L'array spécifie le #theme hook et les variables à passer au template. Le hook est défini via `hook_theme()`, qui relie le hook à un fichier Twig.

— **How would you add the following external JS https ://unpkg.com/@tailwindcss/browse to your controller ?**
Pour ajouter un fichier JavaScript externe dans un controller, il est recommandé de le déclarer dans un fichier `*.libraries.yml` et d'attacher cette library dans le render array du controller via #attached.

— **Given $this->t('Hello, @name!', ['@name' => $username]), what is the search keyword you're gonna use to look for it at /admin/config/regional/-translate ?**

Il faut utiliser la chaîne complète, y compris les placeholders. Pour `$this->t('Hello, @name!', ['@name' => $username])`, le mot-clé à rechercher est : `"Hello, @name!"`.

— **How do you make a string translatable in Drupal javascript files ?**

Dans les fichiers JavaScript de Drupal, les chaînes peuvent être rendues traduisibles avec `Drupal.t()`. Les placeholders peuvent être utilisés via un objet, de manière similaire à `$this->t()` en PHP.

— **Use drush php and get the service path.alias_manager.**

Le service `path.alias_manager` permet de récupérer l'alias d'un nœud. Par exemple, pour `node/1`, on peut obtenir l'alias et construire l'URL complète d'une route avec `Link` et `Url`.

```
[> $alias_manager = \Drupal::service('path_alias.manager');
 = Drupal\path_alias\AliasManager {#1717}
  _
```

FIGURE 2.10 – Service Path Alias Manager

```
$alias_manager = \Drupal::service('path_alias.manager');
$alias = $alias_manager->getAliasByPath('/node/1');

$url_object = Url::fromRoute('entity.node.canonical', ['node' => 1]);
$link = Link::fromTextAndUrl($this->t('View node 1'), $url_object)->toString();
```

```
<div class="weather_page--node-info" style="margin-top: 20px; padding: 10px; border: 1px dashed #ccc;">
  <h2>{{ "Node Information"|t }}</h2>
  <div class="alias">
    <strong>{{ "Alias path:"|t }}</strong> {{ node_alias }}
  </div>
  <div class="link">
    <strong>{{ "Direct link:"|t }}</strong> {{ node_link }}
  </div>
</div>
```

Node Information

Alias path:

Alias: /node/1

Direct link:

Link: View node 1

FIGURE 2.11 – Résultat alias

Chapitre 3

Day 3 : Hooks

Drupal utilise des *hooks*, qui permettent aux modules de modifier ou d'étendre le comportement du cœur ou d'autres modules sans toucher au code source. Un hook est un point précis du système où Drupal appelle automatiquement toutes les implémentations du hook présentes dans les modules activés, généralement nommées `MODULE_NAME_hook_name()`.

Les hooks servent à fournir des informations, modifier des données ou réagir à des événements (ex : modifier un formulaire, ajouter un lien de menu, altérer des données avant affichage). Ils constituent un système clé pour personnaliser Drupal tout en conservant le cœur intact.

3.1. Trouver des Hooks existants

Les hooks sont définis par Drupal core et les modules installés. Pour identifier celui qui convient :

— Consulter la documentation de l'API Drupal, qui détaille chaque hook et ses paramètres.

— Explorer le code source des modules (core ou contrib) pour repérer les appels aux hooks.

Cela permet de choisir rapidement le hook approprié et de l'implémenter dans un module personnalisé.

3.2. Questions

3.2.1. Hooks du module Metatag

```
metatag.api.php U ×
web > modules > contrib > metatag > metatag.api.php
1  <?php
2
3  /**
4   * @file
5   * Document all supported APIs.
6   */
7
8  use Drupal\Core\Routing\RouteMatchInterface;
```

FIGURE 3.1 – Documentation des hooks du module Metatag

Les hooks fournis par le module Metatag permettent aux développeurs de modifier ou d'étendre le comportement de ce module sans toucher à son code source. Ils sont documentés dans le fichier `metatag.api.php`.

Les principaux hooks disponibles sont :

- `hook_metatag_route_entity`
- `hook_metatags_alter`
- `hook_metatags_attachments_alter`
- `hook_metatag_migrate_metatagd7_tags_map_alter`
- `hook_metatag_migrate_nodewordsd6_tags_map_alter`

3.2.2. Hook responsable pour modifier un formulaire

Parameters

array $form: Nested array of form elements that comprise the form.

\Drupal\Core\Form\FormStateInterface $form_state: The current state of the form. The arguments that \Drupal::formBuilder()->getForm() was originally called with are available in the array $form_state->getBuildInfo()['args'].

string $form_id: A string that is the unique ID of the form, set by Drupal\Core\Form\FormInterface::getFormId().

FIGURE 3.2 – Paramètres du hook form_alter

Le hook responsable de la modification d'un formulaire est `hook_form_alter()`.

Il est possible d'étendre un formulaire avec ce hook, par exemple en ajoutant un simple champ texte. On peut le faire en modifiant le tableau $form à l'intérieur de `hook_form_alter()` avant que le formulaire ne soit rendu. Source : La documentation officielle se trouve sur https://api.drupal.org.

3.2.3. Utiliser isFrontPage et hook preprocess

```yaml
path.matcher:
  class: Drupal\Core\Path\PathMatcher
  arguments: ['@config.factory', '@current_route_match']
Drupal\Core\Path\PathMatcherInterface: '@path.matcher'
```

```php
/**
 * {@inheritdoc}
 */
public function isFrontPage() {
  // Cache the result as this is called often.
  if (!isset($this->isCurrentFrontPage)) {
    $this->isCurrentFrontPage = FALSE;
    // Ensure that the code can also be executed when there is no active
    // route match, like on exception responses.
    if ($this->routeMatch->getRouteName()) {
      $url = Url::fromRouteMatch($this->routeMatch);
      $this->isCurrentFrontPage = ($url->getRouteName() && '/' . $url->getInternalPath() === $this->getFrontPagePath());
    }
  }
  return $this->isCurrentFrontPage;
}
```

Preprocess theme variables for templates.

This hook allows modules to preprocess theme variables for theme templates. hook_preprocess_HOOK() can be used to preprocess variables for a specific theme hook.

For more detailed information, see the Theme system overview topic.

Parameters

array $variables: The variables array (modify in place).

string $hook: The name of the theme hook.

FIGURE 3.3 – Hook preprocess

On va utiliser un hook appelé `preprocess` pour envoyer des variables aux templates Twig. Avant cela, on vérifie si la page est la front page grâce à la fonction du service `path.matcher`. Cette vérification sera appliquée pour la page “Weather_page”.

```php
/**
 * Implements hook_preprocess_HOOK() for weather_page.html.twig.
 */
function anytown_preprocess_weather_page(array &$variables) {
  // Récupérer le service path.matcher
  $path_matcher = \Drupal::service('path.matcher');

  // Créer la variable is_front
  $variables['is_front'] = $path_matcher->isFrontPage();
}
```

```twig
  <div>
  <h2>{{"Is this a front page"|t}}</h2>
  {# Affichage conditionnel #}
{% if is_front %}
  <p>This is the front page!</p>
{% else %}
  <p>This is not the front page.</p>
{% endif %}
</div>
</div>
```

Is this a front page

This is the front page!

FIGURE 3.4 – Résultat de la variable is_front

3.2.4. Hook hook_page_attachments_alter

Alter attachments (typically assets) to a page before it is rendered.

Use this hook when you want to remove or alter attachments on the page, or add attachments to the page that depend on another module's attachments (this hook runs after hook_page_attachments(). This hook can be implemented by both modules and themes.

If you try to add anything but #attached and #cache to the array, an exception is thrown.

Parameters

array &$attachments: Array of all attachments provided by hook_page_attachments() implementations.

FIGURE 3.5 – hook_page_attachments_alter

Objectif : ajouter la metatag suivante :

```
<meta name="viewport" content="width=device-width, initial-scale=1, shrink-to-fit=no">
```

```php
function anytown_page_attachments_alter(array &$attachments) {
  // Ajouter une balise meta pour le viewport
  $attachments['#attached']['html_head'][] = [
    [
      '#tag' => 'meta',
      '#attributes' => [
        'name' => 'viewport',
        'content' => 'width=device-width, initial-scale=1, shrink-to-fit=no',
      ],
    ],
    'anytown_viewport_meta', // Un identifiant unique pour éviter les doublons
  ];
}
```

```
<meta name="viewport" content="width=device-width, initial-scale=1, shrink-to-fit=no" />
```

3.2.5. Hook hook_preprocess_menu

function hook_preprocess_HOOK

▸ Same name and namespace in other branches

Preprocess theme variables for a specific theme hook.

This hook allows modules to preprocess theme variables for a specific theme hook. It should only be used if a module needs to override or add to the theme preprocessing for a theme hook it didn't define.

For more detailed information, see the Theme system overview topic.

Parameters

array $variables: The variables array (modify in place).

FIGURE 3.6 – Hook preprocess menu

Objectif : ajouter une classe CSS `.my-custom-class` à tous les éléments du menu.

```php
function anytown_preprocess_menu(array &$variables)
{
    // Vérifie que le tableau contient des items
    if (isset($variables['items']) && is_array($variables['items'])) {
        foreach ($variables['items'] as &$item) {
            // Ajoute la classe CSS à chaque lien du menu
            if (!isset($item['attributes'])) {
                $item['attributes'] = [];
            }
            if (!isset($item['attributes']['class'])) {
                $item['attributes']['class'] = [];
            }
            $item['attributes']['class'][] = 'my-custom-class';
        }
    }
}
```

```
            <li class="my-custom-class menu-item menu-item--collapsed">
<a href="/en/admin/content" title="Find and manage content." id="toolbar-link-system-admin_con
      </li>
        <li class="my-custom-class menu-item menu-item--collapsed">
<a href="/en/admin/structure" title="Administer blocks, content types, menus, etc." id="toolba
      </li>
        <li class="my-custom-class menu-item">
<a href="/en/admin/appearance" title="Select and configure themes." id="toolbar-link-system-th
      </li>
        <li class="my-custom-class menu-item">
<a href="/en/admin/modules" title="Add and install modules to extend site functionality." id="
      </li>
        <li class="my-custom-class menu-item menu-item--collapsed">
<a href="/en/admin/config" title="Administer settings." id="toolbar-link-system-admin_config"
      </li>
        <li class="my-custom-class menu-item">
<a href="/en/admin/people" title="Manage user accounts, roles, and permissions." id="toolbar-l
      </li>
        <li class="my-custom-class menu-item menu-item--collapsed">
<a href="/en/admin/reports" title="View reports, updates, and errors." id="toolbar-link-system
      </li>
        <li class="my-custom-class menu-item">
<a href="/en/admin/help" title="Reference for usage, configuration, and modules." id="toolbar-
      </li>
```

3.2.6. Hook hook_preprocess_block

Objectif : altérer le bloc `system_branding_block` et utiliser le logo suivant : https://static.cdnlogo.com/logos/d/88/drupal-wordmark.svg

```php
function anytown_preprocess_block(array &$variables)
{

    if ($variables['plugin_id'] === 'system_branding_block') {

        $variables['site_logo'] = 'https://static.cdnlogo.com/logos/d/88/drupal-wordmark.svg';

        if (isset($variables['content']['site_logo'])) {
            $variables['content']['site_logo']['#uri'] =
                'https://static.cdnlogo.com/logos/d/88/drupal-wordmark.svg';
        }
    }
}
```

FIGURE 3.7 – Hook preprocess block

```html
<div class="site-branding__inner">  flex
  <a href="/en" rel="home" class="site-branding__logo"> == $0
    <img src="https://static.cdnlogo.com/logos/d/88/drupal-wordmark.svg" alt="Home"
    fetchpriority="high">
  </a>
```

FIGURE 3.8 – Résultat : modification du logo du site

Chapitre 4

Day 04 : Plugins & Forms

4.1. Plugins

Un **plugin** dans Drupal est un petit morceau de fonctionnalité **réutilisable et interchangeable**. Il permet d'ajouter différents comportements sous une même interface.

Les **blocks Drupal** sont des plugins :

— chaque block possède la même structure (titre, contenu, configuration)

— mais chaque module peut créer un block différent

4.1.1. Exemple : HelloWorld Block

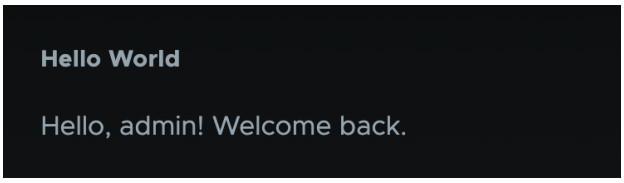
Dans cet exemple, nous utilisons le service `current_user` dans le block `HelloWorld` en utilisant **l'injection de dépendance**.

```php
class HelloWorldBlock extends BlockBase implements ContainerFactoryPluginInterface
{

    private $currentUser;


    public function __construct(array $configuration, $plugin_id, $plugin_definition,  AccountProxyInterface $current_user)
    {
        parent::__construct($configuration, $plugin_id, $plugin_definition);
        $this->currentUser = $current_user;
    }

    public static function create(ContainerInterface $container, array $configuration, $plugin_id, $plugin_definition)
    {
        return new static(
            $configuration,
            $plugin_id,
            $plugin_definition,
            $container->get('current_user')
        );
    }
```



4.2. Forms

Un **Form Controller** est une classe Drupal qui gère un formulaire.

Il contient généralement **trois méthodes principales** :

— `buildForm()` → construit et affiche le formulaire

— `validateForm()` → vérifie les données saisies

— `submitForm()` → traite les données (sauvegarde, message, redirection)

4.2.1. Cycle de vie d'un formulaire Drupal

1. `buildForm()` → Drupal crée et affiche le formulaire

2. L'utilisateur remplit le formulaire et le soumet
3. `validateForm()` → Drupal vérifie les valeurs
4. `submitForm()` → Drupal exécute l'action (par exemple sauvegarder une configuration)

Les valeurs du formulaire sont accessibles via `$form_state`.

4.2.2. Création d'un formulaire

La création d'un formulaire dans Drupal nécessite généralement plusieurs fichiers :

```
anytown.settings:
  path: '/amdin/config/system/anytown'
  defaults:
    _title: 'Anytown Settings'
    _form: 'Drupal\anytown\Form\SettingsForm'
  requirements:
    _permission: 'administer site configuration'
```

```
anytown.settings:
  title: Anytown Settings
  parent: system.admin_config_system
  route_name: anytown.settings
  weight: 10
```

```
/**
 * Configure Anytown settings for this site.
 */
final class SettingsForm extends ConfigFormBase {
```

```
/**
 * {@inheritdoc}
 */
public function validateForm(array &$form, FormStateInterface $form_state): void {
  parent::validateForm($form, $form_state);

  $location = $form_state->getValue('location');
  $value = filter_var($location, FILTER_VALIDATE_INT);
  if (!$value || strlen((string) $value) !== 5) {
    $form_state->setErrorByName('location', $this->t('Invalid ZIP code'));
  }

}

/**
 * {@inheritdoc}
 */
public function submitForm(array &$form, FormStateInterface $form_state) {
  $this->config(self::SETTINGS)
    ->set('display_forecast', $form_state->getValue('display_forecast'))
    ->set('location', $form_state->getValue('location'))
    ->set('weather_closures', $form_state->getValue('weather_closures'))
    ->save();

  $this->messenger()->addMessage($this->t('Anytown configuration updated.'));
```

4.3. Questions

4.3.1. Where can you validate your form data ? (+1 if you mention hooks)

Les données d'un formulaire peuvent être validées dans `validateForm()` du Form Controller ou via des hooks comme `hook_form_alter()` ou `hook_form_FORM_ID_alter()` en ajoutant des fonctions de validation.

4.3.2. How would you render a Form inside a block for a user (not the backoffice form), but in the build method of the block ?

On peut afficher un formulaire dans un block en utilisant le **FormBuilder service** dans la méthode `build()` :

```
return \Drupal::formBuilder()->getForm('Drupal\mymodule\Form\MyForm');
```

Cela retourne le formulaire comme **render array**, qui sera affiché dans le block.

4.3.3. How would you redirect a user after submitting a form ?

On redirige un utilisateur après la soumission d'un formulaire en utilisant `$form_state->setRedirect(` dans `submitForm()` pour indiquer la route cible.

4.3.4. Using #access, how can i hide a field for an anonymous user ?

```
'#access' => $this->currentUser()->isAuthenticated()
```

Dans le render array, cela permet de cacher un champ aux utilisateurs anonymes.

4.3.5. How do you group fields together in a form, just like Field Group module does ?

Dans Drupal, pour regrouper plusieurs champs dans un formulaire, tu peux utiliser un `fieldset` ou un `details element` dans le render array. C'est exactement ce que fait le module **Field Group**.

Chapitre 5

Day 5 : Data types

Dans Drupal, les données sont organisées en plusieurs types :

Contenu : *nodes*, utilisateurs, médias, gérés via l'Entity API. Les entities peuvent avoir des *bundles* (ex. Article, Page) et des champs personnalisés (Field API).

Configuration : paramètres de modules et site, exportables en YAML et versionnables (Configuration API). Les formulaires de configuration héritent de `ConfigFormBase`.

Session et État : données temporaires liées à l'utilisateur ou au site, non exportables.

La Field API permet d'ajouter des champs aux entities. La Configuration API permet de gérer les réglages persistants de manière structurée et transférable entre environnements.

5.1. Configuration API en pratique

La *Configuration API* permet de sauvegarder et lire des paramètres de modules. Exemple : un formulaire de configuration héritant de `ConfigFormBase` lit ou écrit des valeurs stockées dans `anytown.settings`. Cela permet d'avoir des réglages persistants, exportables et importables entre environnements.

Bien que la configuration fonctionne sans schéma, il est recommandé de définir un *Configuration Schema* (`config/schema/*.schema.yml`) pour décrire la structure des données (types, validation, traduction). Des valeurs par défaut peuvent être définies dans `config/install/*.yml` et sont chargées à l'installation du module.

Le *Field API* complète l'Entity API pour les champs personnalisés, tandis que la *Configuration API* structure et versionne les paramètres du site ou des modules.

Questions

Where can you validate your form data ?

Les données d'un formulaire peuvent être validées dans la méthode `validateForm()` du Form Controller ou via des hooks comme `hook_form_alter()` ou `hook_form_FORM_ID_alter()` en ajoutant des fonctions de validation.

Render a form inside a block

On peut afficher un formulaire dans un block en utilisant le service `FormBuilder` dans la méthode `build()`. Cela retourne le formulaire comme render array, prêt à être affiché.

Redirect a user after form submission

On redirige un utilisateur après la soumission en utilisant : `$form_state->setRedirect('route_name'` dans `submitForm()`.

Hide a field for anonymous users using #access

`#access => $this->currentUser()->isAuthenticated()` Permet de cacher un champ aux utilisateurs anonymes.

Group fields in a form (like Field Group)

Utiliser un `fieldset` ou un `details element` dans le render array. Le module Field Group fait exactement cela pour les formulaires et entities.

Dump eu_cookie_compliance config with Drush

Le fichier de configuration se trouve à : `modules/contrib/eu_cookie_compliance/config/install/e`

Why do we need a schema.yml file ?

Le `schema.yml` décrit la structure et les types de la configuration pour que Drupal comprenne ses données et règles.

Load and manipulate a node

- Charger un node : `Node::load($nid)`
- Lire un champ : `$node->get('field_name')->value`
- Modifier un champ : `$node->set('field_name', 'Nouvelle valeur')`
- Sauvegarder le node : `$node->save()`

What is a Constraint ?

Un *Constraint* est une règle de validation d'une donnée ou d'un champ, utilisée dans le Validation API (nodes, fields ou formulaires).

View Builders, teaser and display modes

Le View Builder transforme une entity en render array prêt à être affiché.

— Chaque type d'entité (Node, User, Taxonomy...) a son view builder.

— Il applique le *display mode* (teaser, full, etc.) pour décider quels champs montrer, leur ordre et leurs wrappers.

Entity query and accessCheck

`accessCheck(TRUE)` : seules les entités accessibles par l'utilisateur courant sont retournées. `accessCheck(FALSE)` : toutes les entités correspondant aux conditions sont retournées, utile pour scripts admin ou migrations.

Get a node translation (e.g., French)

`$node->getTranslation('fr')` retourne la traduction en français si le module Content Translation est activé.

Exercice

L'utilisateur peut choisir un node via un formulaire custom. **Code : NodeSelectForm.php**

```php
class NodeSelectForm extends FormBase {

  /**
   * {@inheritdoc}
   */
  public function getFormId() {
    return 'node_select_form';
  }

  /**
   * {@inheritdoc}
   */
  public function buildForm(array $form, FormStateInterface $form_state) {

    $nodes = \Drupal::entityTypeManager()->getStorage('node')->loadByProperties(['type' => 'article']);
    $options = [];
    foreach ($nodes as $node) {
      $options[$node->id()] = $node->getTitle();
    }
```

FIGURE 5.1 – Formulaire NodeSelectForm pour choisir un node.

Le block récupère le node sélectionné via `state` et liste les autres nodes du même type. **Code : SelectedNodeBlock.php**

```php
#[Block(
  id: "selected_node_block",
  admin_label: new TranslatableMarkup("Selected Node Block")
)]
class SelectedNodeBlock extends BlockBase {

  /**
   * {@inheritdoc}
   */
  public function build() {
    $selected_nid = \Drupal::state()->get('anytown_selected_node');

    if (empty($selected_nid)) {
      return [
        '#markup' => $this->t('No node selected yet.'),
        '#cache' => [
          'max-age' => 0,
        ],
      ];
    }
```

FIGURE 5.2 – Block qui affiche le node sélectionné et les autres nodes.

Le thème `anytown_selected_node` reçoit le titre du node sélectionné et la liste des autres nodes. **Code : anytown-selected-node.html.twig**

```html
<div class="selected-node-block">
  <h3>Selected Node:</h3>
  <p>{{ selected_title }}</p>

  {% if other_titles %}
    <h4>Other Nodes of Same Type:</h4>
    <ul>
      {% for title in other_titles %}
        <li>{{ title }}</li>
      {% endfor %}
    </ul>
  {% endif %}
</div>
```

FIGURE 5.3 – Twig template pour afficher le node sélectionné et la liste.

Lien vers le dépôt GitHub

Le code source complet de cet exercice est disponible sur mon dépôt GitHub :

https://github.com/Hibat-Allah-RGUITI/Custom-Module.git